

# 基于 Benders 分解 的 环形生产线平衡： 自顶向下和自底向上方法（算法实现）

郭鹏辉 博士生（Min 团队）

<https://guo.ph>

南京航空航天大学 经济与管理学院

智能决策与风险分析研究所



南京航空航天大学  
NANJING UNIVERSITY OF AERONAUTICS AND ASTRONAUTICS

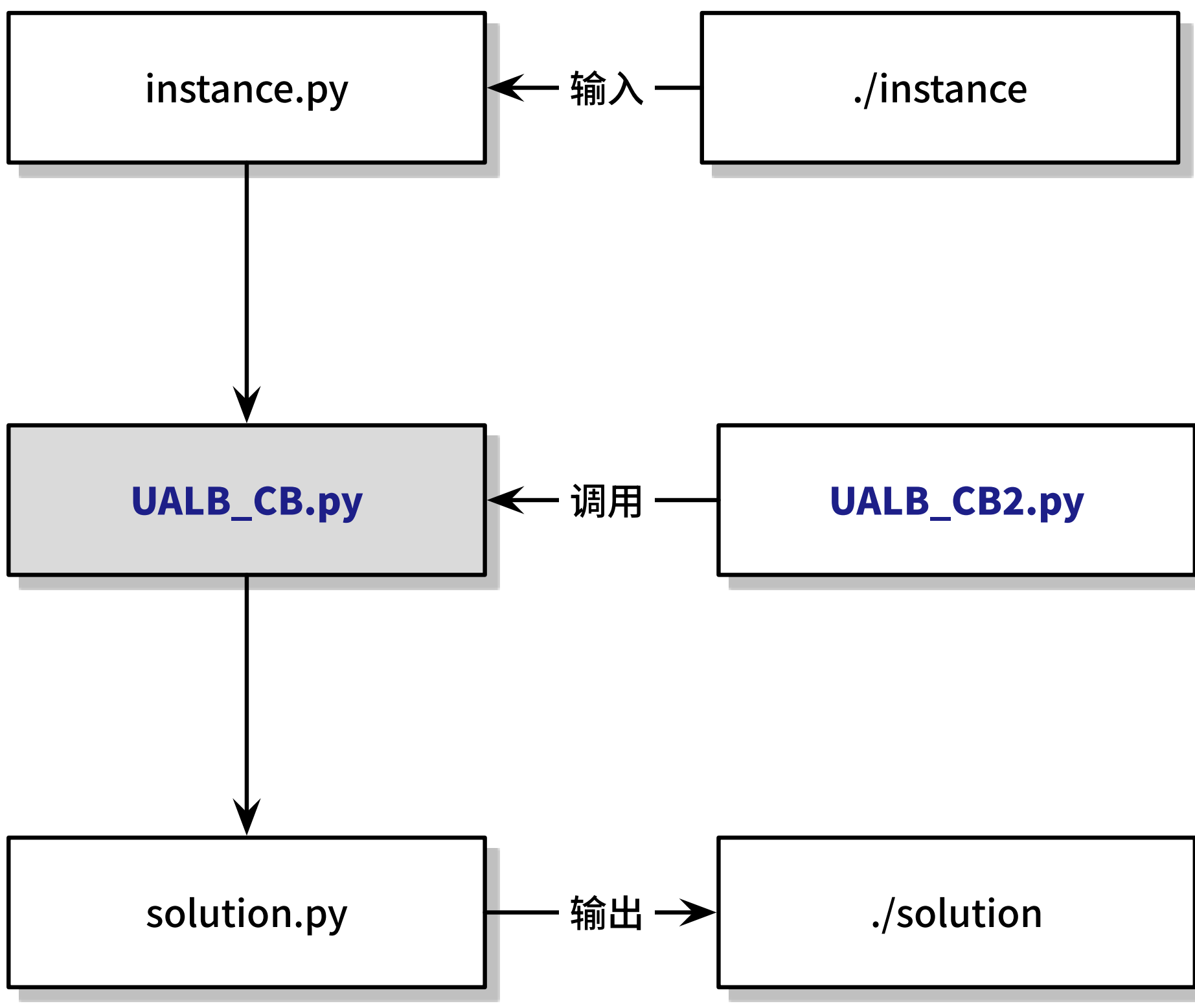
经济与管理学院

2023年9月4日

# 目录

.

|                       |                        |
|-----------------------|------------------------|
| —— README.md          | # 项目部署说明               |
| —— requirements.txt   | # 运行依赖                 |
| —— instances          | # 算例文件夹                |
| —— solutions          | # 求解结果文件夹              |
| —— config.py          | # 运行参数                 |
| —— instance.py        | # 读取和处理算例数据            |
| —— UALB.py            | # 单个 MIP 模型（未参与运算，供参考） |
| —— <b>UALB_CB.py</b>  | # 分解策略（1）自顶向下          |
| —— <b>UALB_CB2.py</b> | # 分解策略（2）自底向上          |
| —— solution.py        | # 验证和保存求解结果            |
| —— utility.py         | # 读取和保存 json 文件        |



# 次要文件

- README.md, 项目简介, 部署和运行说明
- requirements.txt, 依赖的第三方库
- instances, 算例文件夹
- solutions, 求解结果保存文件夹
- config.py, 模型运行参数配置, 除时间外保持默认值即可
- utility.py, 读取和保存 json 文件
- UALB.py, 单个 MIP 模型 (未参与运算, 供参考)



# 主要文件

## instance.py

- **class Station(object), class Process(object), class Worker(object), class Machine(object):** 原样保存官方算例 “instance-\*.txt” 的内容，并作为 Instance.stations, Instance.processes, Instance.workers, Instance.aux\_machines 的值。（注：代码大部分位置的 process 含义应为 “task” 或 “operate”，即“任务”或“工序”，而非其组合 - “程序（process）”。变量命名问题，不影响运行。）
- **class Instance(object):** 对原始的 “instance-\*.txt” 做处理，以方便建模。
  - **\_\_init\_\_(self, instance\_data):** 将原始数据进行处理，以方便建模。
  - **allow\_split():** 根据原始输入判断是否允许拆分工序（未实际使用）。
  - **topological\_ordering(self, processes=None, immediate\_precedence=None):** 计算拓扑序（拓扑序详见“算法设计 - 算法增强 - 快速不可行性证明和增强组合 Benders 割”）。
  - **topological\_ordering\_set(self, processes=None, immediate\_precedence=None):** 计算集合形式的拓扑序（集合形式的拓扑序详见“算法设计 - 算法增强 - 快速不可行性证明和增强组合 Benders 割”）。
  - **make\_aux\_machines(self):** 由于原始的设备有 machine\_type 和 machine\_type\_2 两个属性，该函数合并了这两个属性，简化工序与设备的对应关系。
  - **make\_code\_id\_map(self):** 建立 “instance-\*.txt” 中序号（复杂的 ID）到用于建模的序号（简单的 1, 2, 3, 4）的映射。
  - **get\_immediate\_precedence(self):** 根据 operation\_number 和 joint\_operation\_list 确定工序的紧前工序。
  - **get\_workers\_capable\_processes(self):** 获取工人可以处理的工序集合，获取工序可以分配给的工人集合。
  - **get\_processes\_aux\_machine(self):** 获取工序所需的设备（make\_aux\_machines 函数处理过的）。
  - **get\_stations\_fixed\_aux\_machines(self):** 获取工位的固定设备（make\_aux\_machines 函数处理过的）。
  - **get\_mono\_aux\_machines(self):** 获取独占工位的设备（make\_aux\_machines 函数处理过的）。
  - **get\_stations\_with\_unmovable\_machine(self):** 获取具有和不具有不可移动设备（make\_aux\_machines 函数处理过的）的工位。
  - **\_get\_worker\_capable\_process(self, w):** 获取工人 w 严格兼容（skill capable）和类型兼容（category capable）的工序。
  - **\_get\_efficiency(self, w, p):** 获取工人 w 操作工序 p 的效率。
  - **\_make\_dummy\_process(self, assign\_worker\_to\_process\_vals):** 工序拆分的情况下，根据主问题工人和工序的分配关系，创建虚拟工序。详见“算法设计 - 问题分解（自顶向下）拆分工序 - 子问题的拓展模型”。

# 主要文件

## UALB\_CB.py

- **class Solver(Instance):** 继承 Instance 类，包含建模所需的所有数据。
  - **make\_master\_problem(self, split\_task, obj\_weight=(1, 0, 0), shrink=(0, 0)):** 基于 MIP 创建主问题。工序不可拆分（split\_task=False）和可拆分（split\_task=True）有不同的约束，详见“[算法设计](#)”。obj\_weight 依次为 max 节拍、mean 节拍、和 0 目标函数的权重，实际使用的是 obj\_weight=(0, 0, 1)（见 config.py）。shrink 参数未使用。
  - **make\_cp\_sub\_problem(self, assign\_worker\_to\_process\_vals, split\_task, cycle\_count=None):** 基于 CP 创建子问题。assign\_worker\_to\_process\_vals 为主问题确定的工序和工人的分配关系。工序不可拆分（split\_task=False）和可拆分（split\_task=True）有不同的约束，详见“[算法设计](#)”。cycle\_count 取默认值，即根据官方算例数据允许的最大转圈数。
  - **calc\_station\_lb(self, worker\_to\_process, reverse=False):** 快速子问题不可行性证明。详见“[算法设计 - 算法增强 - 快速不可行性证明和增强组合 Benders 割](#)”。可以根据正向（reverse=False）和反向（reverse=True）的拓扑序计算。
  - **\_\_add\_cb\_cut(self, model, worker\_to\_process, wp\_set):** 根据当前迭代的工人和工序分配关系（worker\_to\_process），向主问题添加组合 Benders 割。可以通过 wp\_set 参数控制 Benders 割所包含的变量范围。详见“[算法设计 - 问题分解（自顶向下）不拆分工序 - 组合 Benders 割](#)”。
  - **\_\_add\_local\_branching\_cut(self, model, worker\_to\_process, left\_or\_right, k):** 根据当前迭代的工人和工序分配关系（worker\_to\_process），向主问题添加局部分支割平面。分为左分支和右分支（left\_or\_right），仅用到右分支。参数 k 控制搜索临域的范围。详见“[算法设计 - 算法增强 - 局部分支](#)”。
  - **\_\_local\_branching(self, model, worker\_to\_process, split\_task):** 局部分支。可兼容工序不可拆分（split\_task=False）和可拆分（split\_task=True）的情况。若找到可行解则停止。局部分支结束时，清空主问题的所有局部分支割（model.local\_branching\_cuts）。
- **solve\_master\_problem(self, model):** 求解主问题。返回主问题不可行或工序和工人的分配关系。
- **solve(self, split\_task, cp\_time\_limit, total\_time\_limit, obj\_weight):** [自顶向下分解的算法主流程](#)。详见“[算法设计 - 算法流程 - Solver.solve\(\)](#)”。可控制是否拆分工序（split\_task）、子问题 CP 求解时间限制（cp\_time\_limit）、总时间限制（total\_time\_limit）、和目标函数权重（obj\_weight）。
- **print\_opt(self, model, worker\_to\_process, process\_to\_station):** 输出找到的可行解或最优解。（注：未能正确显示工序有多个工位和工人的情况，但不影响最终保存的 json 的正确性）。
- **get\_real\_objective(self, master\_model):** 建模时采用的代理目标函数，而非与赛题一致的评价指标，此处计算得到的解的评价指标值。
- **run(self):** 自动选择不拆分工序或拆分工序，自顶向下分解或自底向上分解（调用了 UALB\_CB2.py），输出比赛要求格式的结果数据（调用了 solution.py）。详见“[算法设计 - 整体流程 - Solver.run\(\)](#)”。



# 主要文件

## UALB\_CB2.py

- **class Solver(Instance):** 继承 Instance 类，包含建模所需的所有数据。
  - **make\_master\_problem(self, process\_to\_station):** 确定工人到工序分配的问题。（注：这里为与 UALB\_CB.py 保持一致，将该问题称为 master\_problem，但其地位应为 Benders 分解中的子问题，即接收解和生成割平面。）
  - **make\_cp\_sub\_problem(self, cycle\_count=None):** 确定工序到工位分配的问题。（注：这里为与 UALB\_CB.py 保持一致，将该问题称为 sub\_problem，但其地位应为 Benders 分解中的主问题，即生成解。）
  - **get\_real\_objective(self, master\_model):** 建模时采用的代理目标函数，而非与赛题一致的评价指标，此处计算得到的解的评价指标值。
  - **solve(self, cp\_time\_limit, total\_time\_limit):** 自底向上分解的算法主流程。详见“算法设计 - 问题分解（自底向上）复杂工序拓扑 - 先确定工序排序，再确定工人分配”。可控制问题 CP 求解时间限制（cp\_time\_limit）和总时间限制（total\_time\_limit）。被 UALB\_CB.py 的 Solver.run() 调用。

# 主要文件

## solution.py

- **class Solution(Instance):** 继承 Instance 类，包含验证解正确性的所有数据。（注：默认不验证可行性直接保存）
  - **\_\_init\_\_(self, instance\_data, worker\_to\_process, process\_to\_station, worker\_to\_station, cycle\_num, split\_task):** 根据原始数据（instance\_data，）和求解结果（worker\_to\_process, process\_to\_station, worker\_to\_station）等验证解是否可行。（注：\_\_init\_\_ 函数下内容顺序不能调整。）
  - **valid\_volatility\_rate(self):** 验证节拍波动率约束。
  - **valid\_worker\_capability(self):** 验证工人能力约束。
  - **all\_workers\_are\_assigned(self):** 验证是否有空闲工人。
  - **all\_processes\_are\_assigned(self):** 验证是否有未分配工位的工序。
  - **valid\_station\_worker\_num(self):** 验证每个工位是否至多一个工人。
  - **valid\_worker\_station\_num(self):** 验证每个工人是否都分配了工位，且分配的工位数量不超过上限。
  - **valid\_process\_order(self):** 验证工序顺序约束。
  - **valid\_process\_station\_num(self):** 验证工序拆分是否超过了最大允许工位数量。
  - **valid\_revisit(self):** 验证重复入站约束。
  - **valid\_machine(self):** 验证工位设备约束。
  - **split\_process\_use\_single\_station(self):** 验证被拆分的工序是否占据单个工位。（注：此要求比官方赛题文档要求的约束强。）
  - **write\_solution(self):** 将求解结果按照比赛要求的格式输出。

# 谢谢！

郭鹏辉 博士生（Min 团队）

<https://guo.ph>

南京航空航天大学 经济与管理学院

智能决策与风险分析研究所